

1. Evaluating Arithmetic Expressions

TODO 1: Can pull this bit up into the intro, replacing the old text?

TODO 2: “a traditional pattern” ==> “the traditional pattern of operational semantics”

Our contributions:

- Denotational instead (complete w.r.t. operational – need to prove!)
- Completeness of equational theory w.r.t. denotational semantics
- Treatment of effects
- *Refinement* theory – completeness of *refinement* w.r.t. denotational semantics

TODO 3: Think about rewriting the below without any operational semantics at all – explain the shared bit of the pattern as “most/many/the first few of the chapters start out...”

– “the first few” is a decent idea, since *later* we introduce effects, refinement, and friends

Then the typical thing is:

- opsem
- progress&preservation w.r.t. typing relation
- soundness w.r.t. *observational equivalence*

great people prove *completeness/abstraction/full abstraction* in this world

What we do instead is:

- densem
- soundness w.r.t. densem
- *completeness* w.r.t. densem
- but opsem is subsumed by densem via *interaction trees* and friends!

Or alternatively – follow embedded todos?

Much of this thesis consists of variations on a traditional pattern in the PL literature:

- We introduce a language L , along with some informal intuition and motivation
- We give L a formal syntax – usually parametrized by abstract sets of operations and constants
- We then formalize L ’s “expected” behaviour by equipping it with a simple *operational semantics*
TODO 4: *cut below?* – typically, we will “build up” to a full semantics for L , starting with a simple rewriting-based semantics and then adding features like *state* and *nondeterminism* in stages.
- Then, we define *type theory* for L , consisting of:
 - A *type system* – which defines the types a term can have and the contexts in which they are interpreted. Usually, this will be parametrized by an abstract set of *base types*.

Often, we will study many different languages using the same type system.

- A *typing relation* – this typically consists of *typing rules* which tell us which terms are well-typed in a given context.

TODO 5: *pull this down? or leave here* We usually then want to prove *progress* – that is, that every step in our operational semantics preserves well-typedness.

- An *equational theory* – this gives us a notion of which terms may be considered *equivalent* within a given context

TODO 6: can pull this down – “we then want to prove soundness, but this is *hard* since *observational equivalence* is hard – need “equivalent in every context” and that is pain”

we care about equivalence because we're writing a compiler We usually then want to prove *soundness* – that is, programs which our equational theory identifies as equivalent are “indistinguishable” according to some notion of *observational equivalence*.

TODO 7: denotational semantics – do we pull the operational semantics down?

do we even want it at all –

- **it's not part of the MVP...**
- **but it really helps justify our denotational semantics as correct – otherwise completeness seems particularly abstract**

We'll start by giving a formal account of the simplest practical language – that of *first-order* arithmetic expressions, like $x + y$ and $a + (5 \cdot b) + \sin(3)$.

Rather than complicate our language by introducing binary operators (like $x + y$), unary operators (like $-x$), and n -ary function applications (like $f(x, y, z)$), we can instead normalize everything to *S-expressions* – or *sexprs* – of the form $(f\ e_1 \dots e_n)$. We can then uniformly write:

- binary operations like $x + y$ as binary sexprs $(+ x\ y)$,
- unary operations $-x$ as unary sexprs $(- x)$,
- function applications $f(x, y, z)$ as n -ary sexprs $(f\ x\ y\ z)$.

In particular, the *operator* f always comes first, and there is no order-of-operations – all bracketing is explicit. Restricting ourselves to *first-order* sexprs means that we always require operators f to be atomic symbols drawn from a fixed set O – in particular, we don't support *partial application*, like $((\text{add } 2)\ 3)$, since that would require $f = (\text{add } 2)$ to be a compound expression.

Formally, we define our *syntax* as follows:

Definition 1.1 (First-order S-expressions): fixing

- a set of *variables* $x \in N$
- a set of *operations* $f \in O$
- a set of *constants* $c \in V$

we define the set of *first-order S-expressions* $e \in \text{STm}_N(O)$ to be given by the grammar

$$e ::= x \mid c \mid (f\ e^*)$$

We say a sexpr is *closed* if it contains no free variables – otherwise, it is *open* – where the *free variables* $\text{use}(e)$ of a sexpr e are defined as follows:

$$\text{use}(x) = \{x\} \qquad \text{use}(c) = \{\} \qquad \text{use}((f\ e_1 \dots e_n)) = \bigcup_i \text{use}(e_i)$$

We'll write the set of closed sexprs as $\text{STm}_\emptyset(O)$.

It's easy enough, in the language of simple arithmetic, to define what a closed sexpr “means”, since we can just *evaluate* it to find out.

For example, we might have been taught to evaluate expressions from the “inside out”, like so:

$$(+\ 5\ (\cdot\ 3\ 7)) = (+\ 5\ 21) = 26$$

We can formalize this intuition in terms of a *small-step operational semantics*: given, for each operation $f \in O$, a (partial) function $\text{ev}(f) : V^* \rightarrow V$ telling us what value it returns when called with arguments $c_1, \dots, c_n$, we may define a *reduction relation* $e \rightarrow e'$ on sexprs as follows:

$$\text{step} \frac{\text{ev}(f)(c_1, \dots, c_n) = c}{(f \ c_1 \ \dots \ c_n) \rightarrow c} \quad \text{arg} \frac{c_1, \dots, c_n \in V \quad e_1 \rightarrow e'_1}{(f \ c_1 \ \dots \ c_n \ e_1 \ \dots \ e_m) \rightarrow (f \ c_1 \ \dots \ c_n \ e'_1 \ \dots \ e_m)}$$

Partiality here means that we don't need to assign a meaning to ill-defined expressions like $(/ \ 1 \ 0)$ or $(+)$ (when forced, 0 is a decent choice for both of these) – we can simply leave them as undefined.

Notice that we've fixed the *evaluation order* of sexprs to be left-to-right – that is; we have to finish evaluating all terms to the *left* of a given term before we can get to reducing it.

We say a term e *evaluates* to a constant c – written $e \Downarrow c$ – if there exists a finite sequence of reductions

$$e \rightarrow e_1 \rightarrow \dots \rightarrow e_n \rightarrow c$$

i.e., if $e \rightarrow^* c$ – where R^* denotes the *reflexive, transitive closure* of a relation R .

We call a relation $e \Downarrow c$ a *big-step operational semantics* – as our original intuition for evaluation would suggest, c is a pretty good candidate for the “meaning” of a program $e \Downarrow c$.

In particular, we may define a partial function $\text{ev}(e)$ on closed terms by induction as follows:

$$\text{ev}(c) = c \quad \text{ev}((f \ e_1 \ \dots \ e_n)) = \text{ev}(f)(\text{ev}(e_1), \dots, \text{ev}(e_n))$$

It is straightforward to show that $e \Downarrow \text{ev}(e)$ if and only if $\text{ev}(e)$ is defined.

TODO 8: Consider alternate example: $(\cdot \ (+ \ 1 \ 1) \ x)$

- **Reduces to $(\cdot \ 2 \ x)$ – so these should be “the same”**
- **Intuitively the same as $(+ \ x \ x)$**
- **Intuitively different from $(\cdot \ 3 \ x)$**

We still, however, don't know what an *open* term means: if we try to evaluate

$$(+ \ (+ \ 3 \ 2) \ (\cdot \ x \ 0)) \rightarrow (+ \ 5 \ (\cdot \ x \ 0)) \rightarrow ???$$

the first time we encounter a variable, the relation defined above becomes *stuck* – that is, we reach a term e which is not a constant, and yet does not reduce to anything – so, according to our current conventions, is ill-defined like $(/ \ 1 \ 0)$.

If we fire up a Python REPL and try it, it seems to agree:

```
>>> 5 + (x * 0)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
5 + (x * 0)
      ^
```

```
NameError: name 'x' is not defined
```

On the other hand, if x is in context – say $x = 7$ – this evaluates to 5 as we'd expect:

```
>>> x = 7
```

```
>>> 5 + (0 * x)
```

```
5
```

What this hints at is that the operational semantics of an open term e depends on the context in which it is evaluated – here, on the values of its free variables.

We can model this by defining a *contextual reduction relation* $\gamma \vdash e \rightarrow e'$ parametrized by an *evaluation context* γ – a finitely supported function $\gamma : N \rightarrow V$ assigning a values to variables – as follows:

$$\begin{array}{c} \text{var} \frac{\gamma(x) = c}{\gamma \vdash x \rightarrow c} \qquad \text{step} \frac{\text{ev}(f)(c_1, \dots, c_n) = c}{\gamma \vdash (f \ c_1 \dots c_n) \rightarrow c} \\ \\ \text{arg} \frac{c_1, \dots, c_n \in V \qquad \gamma \vdash e_1 \rightarrow e'_1}{\gamma \vdash (f \ c_1 \dots c_n \ e_1 \dots e_m) \rightarrow (f \ c_1 \dots c_n \ e'_1 \dots e_m)} \end{array}$$

We can then evaluate

$$(x \mapsto 7) \vdash (+ \ (+ \ 3 \ 2) \ (\cdot \ x \ 0)) \rightarrow (+ \ 5 \ (\cdot \ x \ 0)) \rightarrow (+ \ 5 \ (\cdot \ 7 \ 0)) \rightarrow (+ \ 5 \ 0) \rightarrow 5$$

TODO 9: substitution maps open terms to closed terms

TODO 10: discuss:

- algebra: “equivalence under substitution”
- programming: “observational equivalence”

Given a set of *types* $A \in \mathcal{T}$, we’ll define a *stack-typing relation* on O to be any relation

$$f : [A_1, \dots, A_m] \rightarrow [B_1, \dots, B_n]$$

where

- $A_1, \dots, A_m \in \mathcal{T}$ are types – m is called the *input arity* or *arity* of f .
- $B_1, \dots, B_n \in \mathcal{T}$ are types – n is called the *output arity* of f .

For now, we’ll only consider operations with an output arity of 1.

We define a (*typing*) *context* $\Gamma \in \text{Ctx}_N(\mathcal{T})$ to be given by a list of *bindings*

$$x_1 : A_1, \dots, x_n : A_n$$

We can define a simple type system for S-expressions by giving a typing relation $\Gamma \vdash e : A$ that says “under the assumptions of Γ , the S-expression e has type A ”. The rules are:

TODO 11: Typing rules for constants – as distinct from nullary ops

$$\begin{array}{c} \text{var-head} \frac{}{\Gamma, x : A \vdash x : A} \qquad \text{var-tail} \frac{\Gamma \vdash y : B, x \neq y}{\Gamma, x : A \vdash y : B} \\ \\ \text{app} \frac{f : [A_1, \dots, A_n] \rightarrow [B] \quad \forall i. \Gamma \vdash a_i : A_i}{\Gamma \vdash (f \ a_1 \dots a_n) : B} \end{array}$$

TODO 12: Soundness of typing: preservation of typing

TODO 13: Completeness of typing: in the presence of simple types

TODO 14: Church vs. Curry – we will only consider well-typed terms

TODO 15: Doesn’t lose generality – untyping works in this framework due to multi-arity

TODO 16: Substitution and observational equivalence – typing rules for substitution – two open programs are equivalent iff equivalent under every substitution – this is usually where metavariables come in

TODO 17: *Purity* – required for substitution to preserve equivalence – typing rules for purity

TODO 18: Denotational semantics of S-expressions: partial functions

TODO 19: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 20: Equational theory of S-expressions: *purity* – as otherwise you can't *introduce* divisions

TODO 21: Equational theory of S-expressions: *soundness* and *completeness*

2. State

TODO 22: Operational semantics of S-expressions – *state*

TODO 23: *Observational equivalence* in the presence of state

TODO 24: Denotational semantics of S-expressions – the partial state monad

TODO 25: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 26: Equational theory of S-expressions: *purity*

TODO 27: Equational theory of S-expressions: *soundness* and *completeness*